Elastic x AWS UG Mumbai Meetup
April Edition

AWS
User Groups
Mumbai

@newt_sch
*Welcome Everyone

Why Am here?

Credits: hera pheri

Why Am here?

Scaling the Context Engine: Deploying Elastic’s MCP Server on Amazon Bedrock AgentCore Runtime"

Who Am I?

Someshwaran Mohan Kumar

@som23x
blog.someshwaran.me

Developer Advocate

elastic

Elasticsearch Logstash Kibana

Let’s Make this Interactive?

Make a promise( )

Let’s understand Who is Here?

Let’s understand Who is Here?

- Cloud Engineers / AWS Builders / Backend Devs?
- DevOps / Platform / SRE?
- Data Engineers / AIML Engineers?
- Students / beginners exploring AWS or AI?

Here’s what we’ll cover today

- Foundation models with AWS Bedrock
- From stateless agents to memory-aware agents
- Building agents with AWS Strands SDK (access to tools)
- Running agents with AWS AgentCore and its Resources
- Short-term memory, long-term memory, and RAG
- Connecting tools via Elastic MCP and Elastic Agent Builder
- A Quick Demo

Credits: carryminati

Quick Question

What do you think an LLM is…? and how is it different from an Agent?

How we know LLMs?

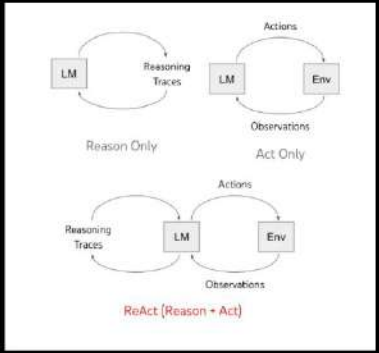
LLM is like Uday Shetty

*Introducing Agents

Agent is like Majnu Bhai!

ReAct: Synergizing Reasoning and Acting in Language Models

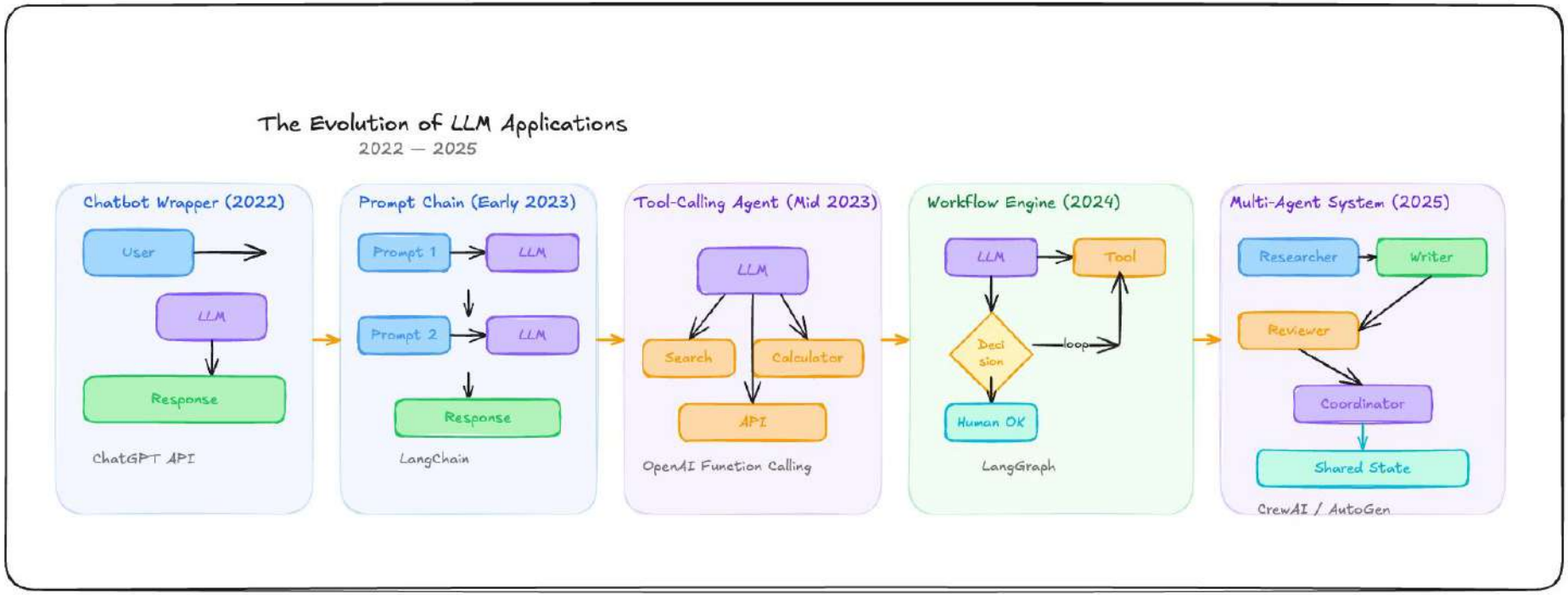
November 8, 2022 ·
Posted by Shunyu Yao, Student Researcher, and Yuan Cao, Research Scientist, Google Research, Brain Team



Recent advances have expanded the applicability of language models (LM) to downstream tasks. On one hand, existing language models that are properly prompted, via chain-of-thought, demonstrate emergent capabilities that carry out self-conditioned reasoning traces to derive answers from questions, excelling at various arithmetic, commonsense, and symbolic reasoning tasks. However, with chain-of-thought prompting, a model is not grounded in the external world and uses its own internal representations to generate reasoning traces, limiting its ability to reactively explore and reason or update its knowledge. On the other hand, recent work uses pre-trained language models for planning and acting in various interactive environments (e.g., text games, web navigation, embodied tasks, robotics), with a focus on mapping text contexts to text actions via the language model's internal knowledge. However, they do not reason abstractly about high-level goals or maintain a working memory to support acting over long horizons.

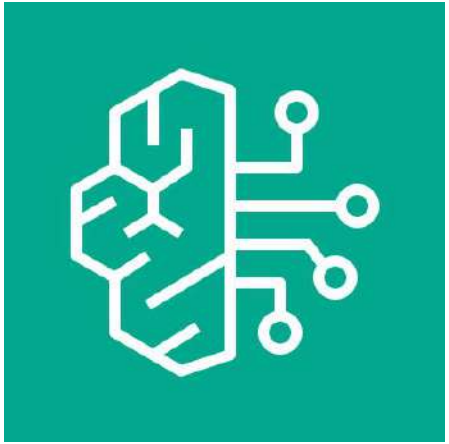
In "ReAct: Synergizing Reasoning and Acting in Language Models", we propose a general paradigm that combines reasoning and acting advances to enable language models to solve various language reasoning and decision making tasks. We demonstrate that the *Reason+Act* (ReAct) paradigm systematically outperforms reasoning and acting only paradigms, when prompting bigger language models and fine-tuning smaller language models. The tight integration of reasoning and acting also presents human-

Source: https://research.google/blog/react-synergizing-reasoning-and-acting-in-language-models/


The Evolution of LLM Applications
2022 — 2025
Chatbot Wrapper (2022)
User
LLM
Response
ChatGPT API
Prompt Chain (Early 2023)
Prompt 1
LLM
Prompt 2
LLM
Response
LangChain
Tool-Calling Agent (Mid 2023)
LLM
Search
Calculator
API
OpenAI Function Calling
Workflow Engine (2024)
LLM
Tool
Deci sion
loop
Human OK
LangGraph
Multi-Agent System (2025)
Researcher
Writer
Reviewer
Coordinator
Shared State
CrewAI / AutoGen

How to use these LLMs or build Agents?

Conversation Layer + Infra Layer

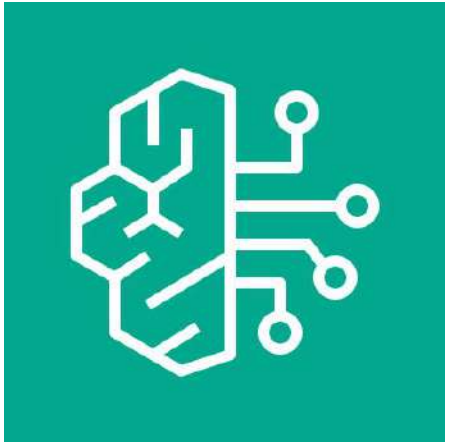


AWS Bedrock
(For LLMs)

Knowledge + Context Layer

Now, we have 🧠, how building Agents?

Conversation Layer
+ Infra Layer



AWS Bedrock
(For LLMs)

STRANDS AGENTS

Tools, Functions

Knowledge +
Context Layer

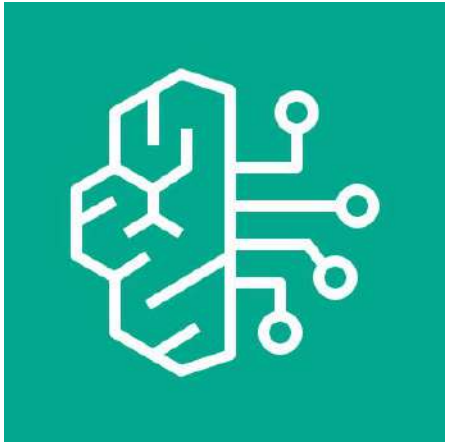
Why Strands Agents? What does it solve?

*Why it's exist?

Agents are built, what’s NEXT?

@newt_sch
*In Production
bilkul ricks nai lene ka...

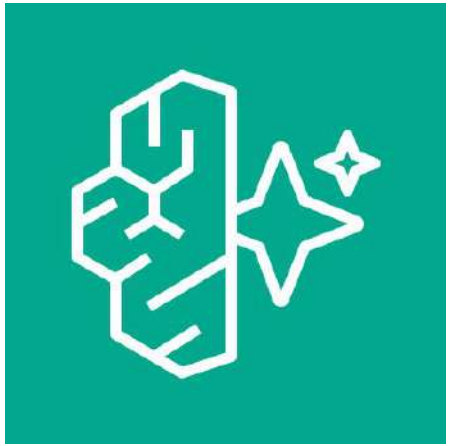
Conversation Layer
+ Infra Layer



AWS Bedrock
(For LLMs)

STRANDS AGENTS

Tools, Functions



AWS Agentcore
(Infrastructure)

Knowledge +
Context Layer

AWS AgentCore: The Infra Layer

YOUR STRANDS AGENT

AgentCore Runtime Resources

Gateway	Secure API routing and traffic management
Identity	Agent identity, authentication, and authorization
Policy	Guardrails, content filtering, and compliance controls
Evaluations	Quality measurement and performance tracking

Introduction to the Goldfish Problem

What is a Goldfish Problem?

*LLMs/ Agents

Ghajini Problem

Agents without memory reset to zero. Every. Single. Session.

Session Starts

User provides full context. Goals, preferences, history — everything.

Agent Responds

Works perfectly. Uses context. Delivers value.

Session Ends

Everything forgotten. Context gone. Memory wiped.

The result: Users repeat themselves endlessly. Agents can never learn preferences. No continuity, no personalization, no compounding value.

It’s a Context Problem and it’s an Agent Problem

(How is it solved?)

AWS AgentCore: The Infra Layer

YOUR STRANDS AGENT

AgentCore Runtime Resources

Gateway	Secure API routing and traffic management
Identity	Agent identity, authentication, and authorization
Policy	Guardrails, content filtering, and compliance controls
Evaluations	Quality measurement and performance tracking
Memory	Session Summarizer · Analyst Preferences · Data Insights

Strands SDK + Code Walkthrough

```
src/main.py — Agent Entrypoint

from strands import Agent
from bedrock_agentcore.runtime import BedrockAgentCoreApp

app = BedrockAgentCoreApp()

@app.entrypoint
async def invoke(payload, context):
    agent = Agent(
        model = bedrock_model,
        tools = mcp_client.tools + es_tools,
        system_prompt = ANALYST_PROMPT,
        session_manager = memory_manager
    )
    response = agent(payload['query'])
    return { 'response': response }
```

`@app.entrypoint`

Wraps your function as an AgentCore-compatible handler. The Runtime calls this for every invocation.

`MCP Tools + ES Tools`

Elastic MCP tools (query, search, mappings) combined with optional ES semantic memory tools passed as a single list.

`session_manager`

Injects AgentCore Memory into the Strands Agent. Memory is automatically loaded before and saved after each turn.

`agentcore launch`

Single CLI command that packages, deploys, and provisions the HTTPS endpoint. No Docker, no ECR needed.

What’s Left?

Conversation Layer
+ Infra Layer

STRANDS AGENTS

Tools, Functions

AWS Bedrock
(For LLMs)

AWS Agentcore
(Infrastructure)

Knowledge +
Context Layer

elastic

RAG (MCP +
Agent Builder)

Elasticsearch: The Context Layer

Not competing with AgentCore Memory. Completing it.

- RAG over live operational data
- Hybrid BM25 + vector search
- Semantic insight storage

Knows the DOMAIN

Together: An agent that remembers who you are AND knows what’s actually happening in your systems right now.

Elasticsearch: The Context Layer

Not competing with AgentCore Memory. Completing it.

Elasticsearch

- RAG over live operational data
- Hybrid BM25 + vector search
- Weblogs, orders, events now
- Semantic insight storage
- Kibana visibility + control

Knows the DOMAIN

AgentCore Memory

- Episodic memory
- Session summaries
- User preferences
- Conversation history
- AWS-managed, automatic

Remembers the CONVERSATION

Together: An agent that remembers who you are AND knows what’s actually happening in your systems right now.

How to consume this Context?

Elasticsearch: MCP + Agent Builder

What’s MCP?

Why MCP Exists?

*Before

Python

```python
from elasticsearch import Elasticsearch

es = Elasticsearch(
    "https://my-elastic.example.com",
    api_key="MY_API_KEY"
)

def search_logs(error_code: str):
    return es.search(
        index="weblogs-*",
        query={
            "match": {
                "status": error_code
            }
        }
    )

results = search_logs("500")
print(results)
```

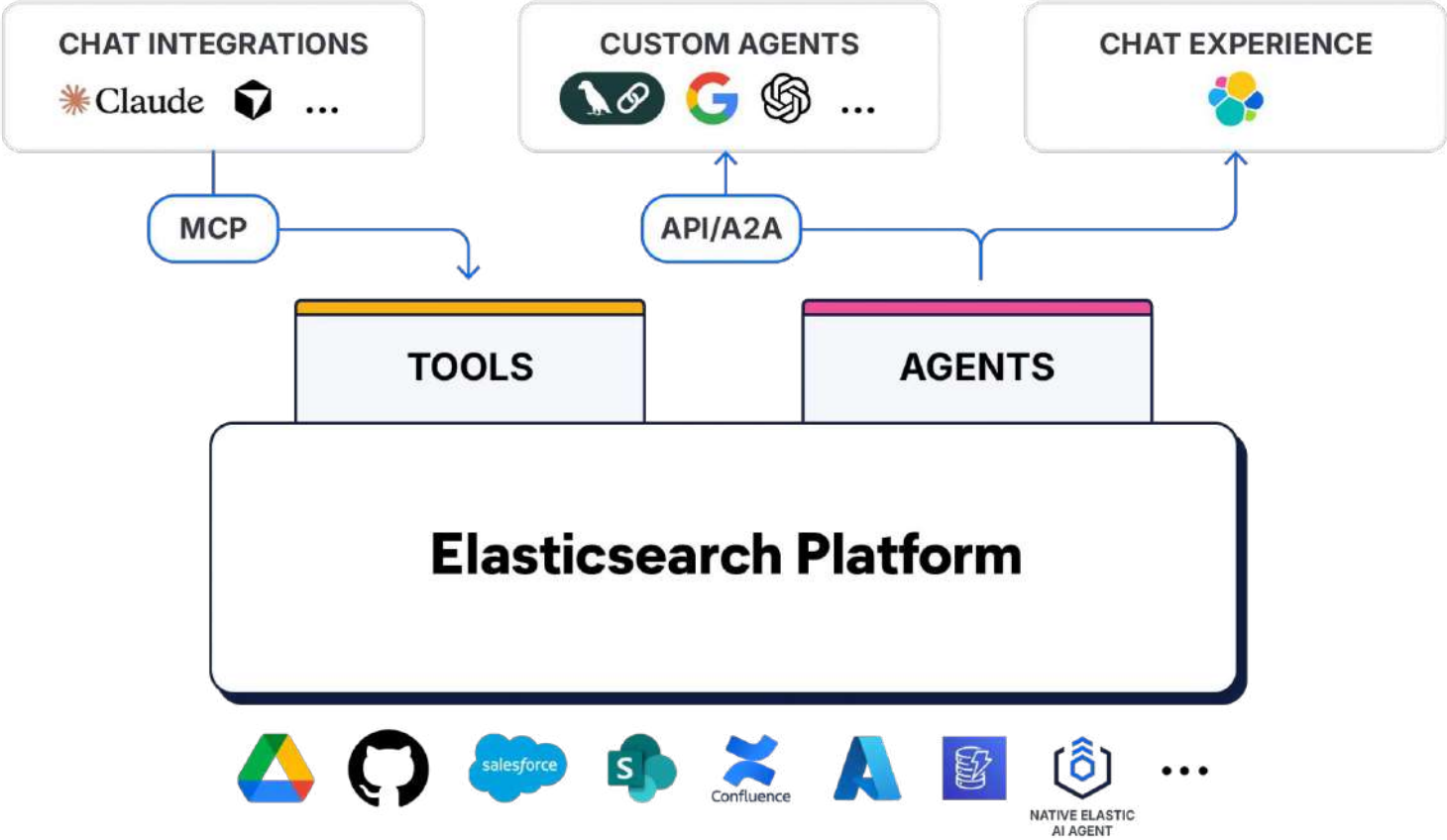
*After

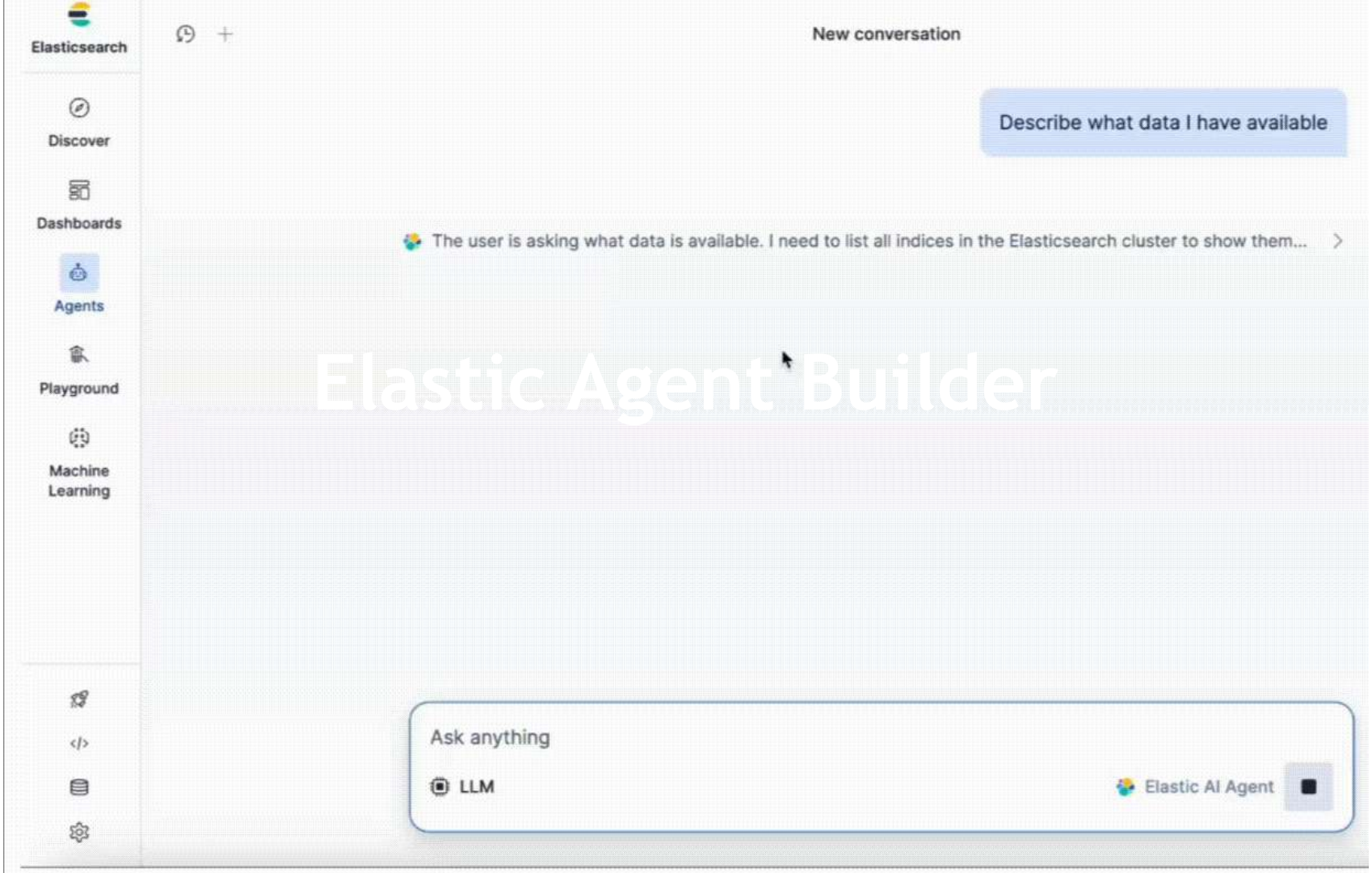
Python

```python
from strands import Agent

agent = Agent(
    model=bedrock_model,
    tools=mcp_client.tools   # discovered from Elastic MCP server
)

response = agent("""
Find the top 5 countries generating 5xx errors this week
and show the trend over time.
""")

print(response)
```

Exposed Tools:

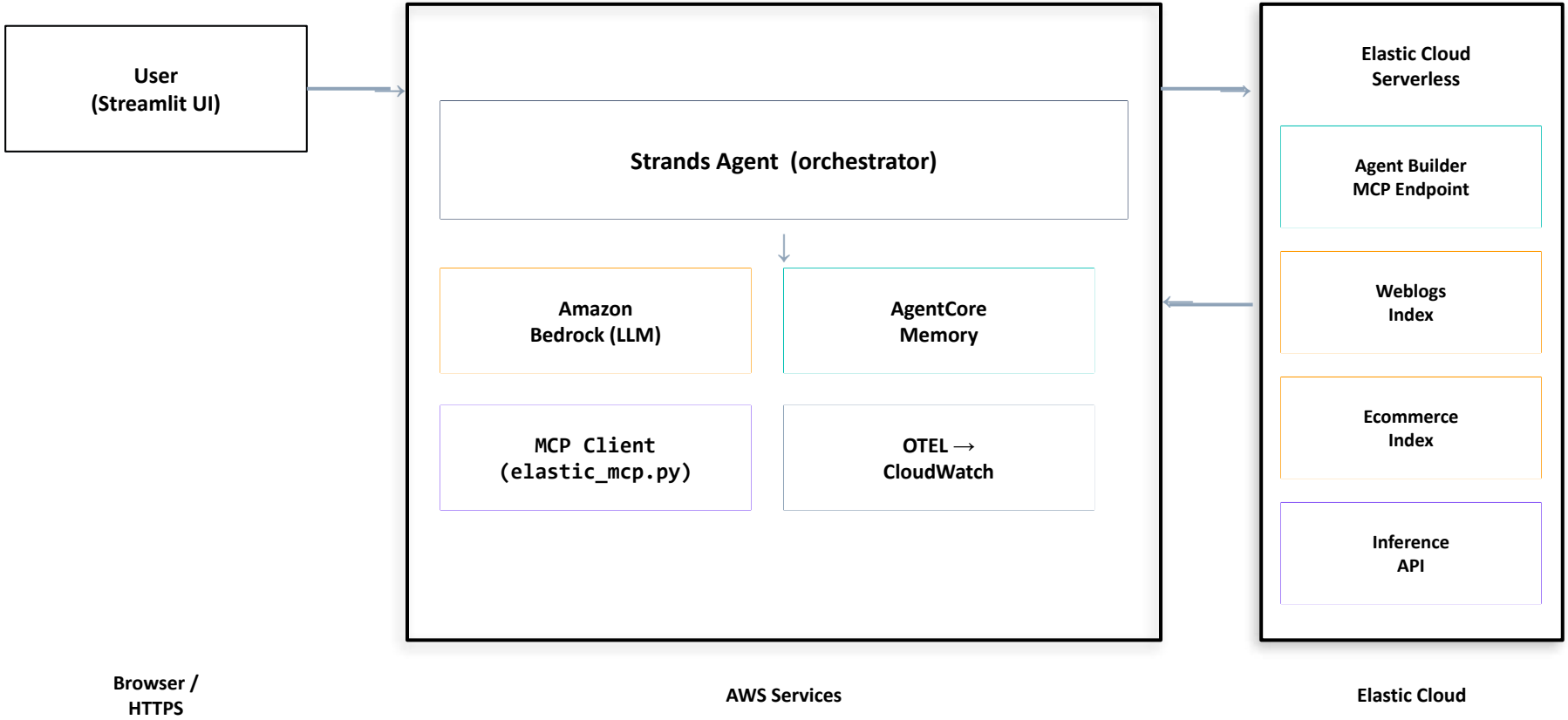
- execute_esql
- search_index
- get_index_mappings
- get_indices
- get_shard_info


CHAT INTEGRATIONS
Claude
...
CUSTOM AGENTS
...
CHAT EXPERIENCE
MCP
API/A2A
TOOLS
AGENTS
Elasticsearch Platform
salesforce
Confluence
NATIVE ELASTIC AI AGENT
...


Elasticsearch
Discover
Dashboards
Agents
Playground
Machine Learning
New conversation
Describe what data I have available
The user is asking what data is available. I need to list all indices in the Elasticsearch cluster to show them...
Elastic Agent Builder
Ask anything
LLM
Elastic AI Agent

Architecture Deep Dive

How all the pieces connect



DEMO

Strands Agent + AgentCore + Elasticsearch MCP

Step 1: Github Repo

elastic-agentcore Public

Pin | Watch 0 | Fork 0 | Star 0

main | 1 Branch | 2 Tags | Go to file | Add file | Code

Som23Git Add optional Elasticsearch-as-memory tools (v2.0.0) 16d25ee · 2 weeks ago 3 Commits

.devcontainer	Add GitHub Codespaces devcontainer and setup script	2 weeks ago
scripts	Add optional Elasticsearch-as-memory tools (v2.0.0)	2 weeks ago
src	Add optional Elasticsearch-as-memory tools (v2.0.0)	2 weeks ago
.env.example	Add optional Elasticsearch-as-memory tools (v2.0.0)	2 weeks ago
.gitignore	Elastic Data Analyst Agent on Amazon Bedrock AgentCore	2 weeks ago
.python-version	Elastic Data Analyst Agent on Amazon Bedrock AgentCore	2 weeks ago
README.md	Add optional Elasticsearch-as-memory tools (v2.0.0)	2 weeks ago
app.py	Add optional Elasticsearch-as-memory tools (v2.0.0)	2 weeks ago
main.py	Elastic Data Analyst Agent on Amazon Bedrock AgentCore	2 weeks ago
pyproject.toml	Elastic Data Analyst Agent on Amazon Bedrock AgentCore	2 weeks ago
requirements.txt	Elastic Data Analyst Agent on Amazon Bedrock AgentCore	2 weeks ago
setup.sh	Add GitHub Codespaces devcontainer and setup script	2 weeks ago
uv.lock	Elastic Data Analyst Agent on Amazon Bedrock AgentCore	2 weeks ago

README

Elastic Data Analyst Agent on Amazon Bedrock AgentCore

An AI-powered data analyst agent that queries your Elasticsearch weblogs and ecommerce data using natural language. Built with the **Strands Agents SDK**, powered by **Amazon Bedrock** foundation models, connected to **Elastic Cloud Serverless** via the hosted **Agent Builder MCP** endpoint, with **AgentCore Memory** for session continuity and long-term personalization.

About

Strands agent on Amazon Bedrock AgentCore that queries Elastic Cloud Serverless (weblogs & ecommerce) via the hosted Agent Builder MCP, with AgentCore Memory, optional Elasticsearch-backed memory tools, and a Streamlit demo UI.

Readme
Activity
0 stars
0 watching
0 forks

Releases

2 tags

Create a new release

Contributors 1

Som23Git Som23Git

Languages

Python 96.9% Shell 3.1%

Suggested workflows

Based on your tech stack

Python package Configure

Create and test a Python package on multiple Python versions.

Python Package Configure

Step 2: Agentcore Memory

Step 3: Elastic Agent Builder

Step 4: Streamlit App

Let’s Connect Again!
jaa raha hu lekin wapas
jarur aaunga